

PYTHON COLLECTIONS PLAYBOOK

Chapter 2 — Container Data Types & Custom Collections

Lists, Tuples, Sets, Dictionaries — and extending them with the collections module

SECTION 1 — THEORY

1.1 Container Data Types — An Overview

In Python, a container is any object that can hold references to other objects. Containers are the fundamental building blocks of virtually every program — from a simple script that collects user input to a distributed ML pipeline that processes millions of records.

Python's four built-in container types each exist because they solve a different category of problem:

- Lists: when you need an ordered, changeable sequence of items — the most general-purpose container.
- Tuples: when you need an ordered sequence that must never change — useful for data integrity and performance.
- Sets: when you need a collection of unique items and fast membership testing — ideal for deduplication.
- Dictionaries: when you need to associate keys with values for fast lookups — the backbone of JSON-like data structures.

All four are heterogeneous — they can hold objects of any type simultaneously — and all are iterable, meaning you can loop over them with a for loop.

Quick Overview of All Four Containers
List — ordered, mutable, allows duplicates
numbers = [1, 2, 3]
Tuple — ordered, immutable, allows duplicates
person = ("John", 25)
Set — unordered, mutable, NO duplicates
colors = {"red", "blue", "green"}
Dictionary — key-value pairs, keys are unique
student = {"name": "Alice", "grade": "A"}

1.2 Lists — Ordered, Mutable Sequences

A list is Python's most versatile container. It is an ordered collection of items, meaning the position (index) of each element is defined and preserved. Lists are mutable, which means you can add, remove, or modify elements after the list has been created.

Lists are ideal for situations where the order of items matters, the collection will change over time, or you need to access elements by their position. In data science, lists are used constantly — for holding feature

names, storing API results before converting to a DataFrame, accumulating processed records in a loop, and much more.

Key characteristics of lists:

- Created with square brackets: `my_list = [1, 'hello', 3.14, True]`
- Zero-indexed: the first element is at index 0, the last at index -1.
- Ordered: the sequence of insertion is preserved.
- Mutable: you can change, add, or remove elements at any time.
- Allow duplicates: the same value can appear multiple times.
- Heterogeneous: a single list can hold integers, strings, booleans, and even other lists.

List Methods — Complete Reference

Method / Operation	Description	Example
<code>append(x)</code>	Add item x to the end of the list	<code>lst.append(5)</code>
<code>insert(i, x)</code>	Insert item x at index i, shifting others right	<code>lst.insert(1, 'a')</code>
<code>remove(x)</code>	Remove the first occurrence of value x	<code>lst.remove(3)</code>
<code>pop(i)</code>	Remove and return item at index i (default: last)	<code>lst.pop(0)</code>
<code>sort()</code>	Sort in place (ascending by default)	<code>lst.sort()</code>
<code>sort(reverse=True)</code>	Sort in descending order	<code>lst.sort(reverse=True)</code>
<code>reverse()</code>	Reverse the list in place	<code>lst.reverse()</code>
<code>index(x)</code>	Return index of first occurrence of x	<code>lst.index('a')</code>
<code>count(x)</code>	Count how many times x appears	<code>lst.count(3)</code>
<code>extend(iter)</code>	Append all items from an iterable to the list	<code>lst.extend([6, 7])</code>
<code>clear()</code>	Remove all items from the list	<code>lst.clear()</code>
<code>copy()</code>	Return a shallow copy of the list	<code>new = lst.copy()</code>
<code>len(lst)</code>	Return the number of items in the list	<code>len(lst)</code>
<code>lst[i:j]</code>	Slice from index i up to (not including) j	<code>lst[1:4]</code>

1.3 List Comprehensions — Pythonic Data Transformation

A list comprehension is a concise, expressive way to create a new list by applying an expression to each item in an existing iterable, optionally filtering items with a condition. It is one of the most distinctly Pythonic features of the language and is widely used in data processing pipelines.

The general syntax is:

List Comprehension Syntax
Basic form
<code>new_list = [expression for item in iterable]</code>
With filter condition
<code>new_list = [expression for item in iterable if condition]</code>
With transformation and filter
<code>new_list = [transform(item) for item in iterable if predicate(item)]</code>

Why prefer list comprehensions over explicit for-loops?

- **Performance:** List comprehensions are executed in C internally by CPython, making them faster than equivalent for-loops with `.append()` calls.
- **Readability:** A well-written comprehension expresses intent in a single line, reducing cognitive overhead.
- **Functional style:** They encourage thinking in terms of transformations and filters rather than imperative mutation.

Data Science Use Case

List comprehensions are extensively used when pre-processing data — for example: `[float(x) for x in raw_values if x != 'N/A']` converts a list of raw strings to floats while skipping missing values, all in one readable line.

1.4 Tuples — Ordered, Immutable Sequences

A tuple is superficially similar to a list — it is ordered, allows duplicates, and supports indexing and slicing. The critical difference is immutability: once a tuple is created, you cannot add, remove, or change any of its elements. Attempting to do so raises a `TypeError` at runtime.

This immutability is a feature, not a limitation. It gives tuples several important advantages:

- **Safety:** If a function returns a tuple, the caller knows the data cannot be accidentally mutated.
- **Performance:** Tuples are allocated more efficiently in memory than lists and are faster to create and iterate.
- **Hashability:** Because tuples are immutable, they can be used as dictionary keys or stored in sets (if their contents are also immutable). Lists cannot.
- **Semantic clarity:** A tuple communicates 'this is a fixed record' rather than 'this is a collection that will change'.

Common use cases for tuples:

- Returning multiple values from a function: `return (result, confidence_score, elapsed_time)`

- Representing fixed records: `coordinates = (latitude, longitude)`, `RGB = (255, 128, 0)`
- Dictionary keys that group multiple values: `cache[(user_id, query)] = result`
- Named tuples for lightweight, readable data classes (`collections.namedtuple`).

Tuple Packing & Unpacking

Python automatically packs multiple values into a tuple when separated by commas. Unpacking (`name, age = person`) extracts those values back into individual variables. This is the mechanism behind the elegant Python idiom: `a, b = b, a` to swap two variables without a temporary variable.

1.5 Sets — Unordered Collections of Unique Values

A set is a container that stores only unique elements and provides no guaranteed ordering. Internally, sets are implemented as hash tables — the same underlying data structure as dictionary keys — which gives them extremely fast membership testing (the 'in' operator runs in $O(1)$ average time, compared to $O(n)$ for lists).

When to use sets over lists:

- Deduplication: Converting a list to a set instantly removes all duplicates: `unique_emails = set(all_emails)`
- Membership testing: 'is this value in the collection?' is dramatically faster with a set than a list.
- Mathematical set operations: union, intersection, difference, and symmetric difference — all built-in.

Set Operations — Complete Reference

Method / Operation	Description	Example
<code>add(x)</code>	Add element <code>x</code> to the set	<code>s.add(4)</code>
<code>remove(x)</code>	Remove <code>x</code> ; raises <code>KeyError</code> if not found	<code>s.remove(3)</code>
<code>discard(x)</code>	Remove <code>x</code> if present; no error if absent	<code>s.discard(99)</code>
<code>pop()</code>	Remove and return an arbitrary element	<code>s.pop()</code>
<code>clear()</code>	Remove all elements	<code>s.clear()</code>
<code>union(s2)</code>	All elements from both sets (<code>s s2</code>)	<code>s.union(s2)</code>
<code>intersection(s2)</code>	Only elements in both sets (<code>s & s2</code>)	<code>s.intersection(s2)</code>
<code>difference(s2)</code>	Elements in <code>s</code> but not in <code>s2</code> (<code>s - s2</code>)	<code>s.difference(s2)</code>
<code>symmetric_difference(s2)</code>	Elements in either set but not both (<code>s ^ s2</code>)	<code>s.symmetric_difference(s2)</code>

<code>issubset(s2)</code>	True if all elements of <code>s</code> are in <code>s2</code>	<code>s.issubset(s2)</code>
<code>issuperset(s2)</code>	True if <code>s</code> contains all elements of <code>s2</code>	<code>s.issuperset(s2)</code>
<code>isdisjoint(s2)</code>	True if <code>s</code> and <code>s2</code> have no elements in common	<code>s.isdisjoint(s2)</code>

1.6 Dictionaries — Key-Value Mappings

A dictionary is Python's implementation of a hash map — a data structure that associates unique keys with values, enabling $O(1)$ average-time lookups regardless of the dictionary's size. Dictionaries are one of the most important data structures in Python and are the direct Python equivalent of JSON objects.

Since Python 3.7, dictionaries preserve insertion order, combining the fast lookup of a hash map with the predictable ordering of a sequence.

Key rules and characteristics:

- Keys must be unique — assigning a value to an existing key overwrites it.
- Keys must be hashable (immutable): strings, numbers, and tuples are valid keys; lists are not.
- Values can be any Python object: integers, strings, lists, other dictionaries, functions, class instances.
- Created with curly braces and colons: `{'key': value}` or using `dict(key=value)`.

Dictionary Methods — Complete Reference

Method / Operation	Description	Example
<code>d[key]</code>	Access value for key; raises <code>KeyError</code> if absent	<code>d['name']</code>
<code>d.get(key, default)</code>	Access value; returns default if key absent	<code>d.get('age', 0)</code>
<code>d[key] = value</code>	Add new key-value pair or update existing	<code>d['city'] = 'Delhi'</code>
<code>d.pop(key)</code>	Remove and return value for key	<code>d.pop('age')</code>
<code>d.keys()</code>	Returns a view of all keys	<code>list(d.keys())</code>
<code>d.values()</code>	Returns a view of all values	<code>list(d.values())</code>
<code>d.items()</code>	Returns a view of (key, value) tuples	<code>for k, v in d.items()</code>
<code>d.update(d2)</code>	Merge <code>d2</code> into <code>d</code> , overwriting existing keys	<code>d.update({'x': 1})</code>
<code>d.setdefault(k, v)</code>	Set key to <code>v</code> only if key is not already present	<code>d.setdefault('hits', 0)</code>

<code>d.copy()</code>	Return a shallow copy of the dictionary	<code>copy = d.copy()</code>
<code>len(d)</code>	Return number of key-value pairs	<code>len(d)</code>
<code>key in d</code>	Test membership of key ($O(1)$)	<code>'name' in d</code>

1.7 Choosing the Right Container — Comparison Reference

Selecting the appropriate container type for each situation is one of the most impactful decisions you make when writing Python code. The wrong choice can lead to buggy code (using a list when uniqueness is required), poor performance (using a list for membership testing instead of a set), or unnecessary complexity (using a dictionary when a tuple suffices).

Property	List	Tuple	Set	Dictionary
Ordered?	Yes	Yes	No	Keys: Yes
Mutable?	Yes	No	Yes	Yes
Duplicates?	Yes	Yes	No	Keys: No
Syntax	<code>[]</code>	<code>()</code>	<code>{ }</code>	<code>{'k': v}</code>
Index access?	Yes	Yes	No	By key
Best for	Dynamic sequences	Fixed data	Uniqueness	Mappings
Hash-based lookup?	No	No	Yes	Yes
Memory usage	Moderate	Low	Moderate	Moderate–High

Decision guide:

- Use a list when order matters and the data will change — processing pipeline stages, accumulated results, feature vectors.
- Use a tuple when the data is fixed and its position has meaning — function return values, database rows, coordinate pairs.
- Use a set when uniqueness is the primary concern — deduplication, tag collections, fast membership tests.
- Use a dictionary when you need to look things up by name or identifier — configuration settings, API response parsing, feature stores, caches.

1.8 The collections Module — Extending Built-In Types

The collections module in Python's standard library provides specialized container data types that extend the behaviour of the four built-in containers. Rather than reimplementing data structures from scratch, the module gives you higher-level abstractions for common patterns.

The three `UserX` classes in `collections` — `UserList`, `UserDict`, and `UserString` — are wrapper classes designed specifically for subclassing. While it is technically possible to inherit directly from `list`, `dict`, or `str`, this approach has subtle pitfalls: some built-in methods on those classes do not call overridden methods in subclasses, leading to inconsistent behaviour. The `UserX` wrappers solve this by delegating all operations through Python-level method calls, so your overrides are always respected.

Other notable classes in the `collections` module:

- `namedtuple`: Creates tuple subclasses with named fields — useful for readable, lightweight records.
- `defaultdict`: A dictionary that returns a default value for missing keys instead of raising `KeyError`.
- `Counter`: A dictionary subclass for counting hashable objects — perfect for word frequencies, tag counts, etc.
- `OrderedDict`: A dictionary that explicitly tracks insertion order (though regular dicts do this since Python 3.7).
- `deque`: A double-ended queue that supports $O(1)$ appends and pops from both ends — more efficient than a list for queue/stack operations.

When to Use `UserList` vs. list Subclass

Always prefer `UserList` (and `UserDict`, `UserString`) when creating custom container classes. Inheriting directly from `list`, `dict`, or `str` can cause methods like `update()` or `extend()` to bypass your overrides. `UserX` classes store data in a `self.data` attribute and route all operations through standard Python method dispatch, guaranteeing your custom behaviour is always invoked.

1.9 `UserList` — Custom List Classes

`UserList` is a wrapper class that behaves exactly like a regular Python list, but is designed to be subclassed safely. Your subclass stores its data in a `self.data` attribute (a regular list), and you override specific methods to add custom behaviour — typically validation logic that restricts what can be stored.

Every method from the regular list interface is available: `append`, `extend`, `insert`, `remove`, `pop`, `sort`, `index`, `count`, slicing, iteration, and `len()`. You override only the methods you want to customize; all others work as expected.

Typical patterns for `UserList` subclasses:

- Type-restricted lists: raise `TypeError` if an item of the wrong type is appended.
- Range-restricted lists: raise `ValueError` if a numeric value is out of bounds.
- Logged lists: write an audit log every time an item is added or removed.
- Capped lists: automatically discard the oldest item when a maximum size is reached (a sliding window).

1.10 `UserDict` — Custom Dictionary Classes

UserDict wraps a regular dictionary stored in `self.data` and allows you to override any dictionary operation. The most commonly overridden method is `__setitem__`, which is called whenever a key-value pair is added or updated. This makes UserDict ideal for enforcing conventions on keys or values at write time.

Practical applications of UserDict subclasses:

- Key normalization: automatically convert all keys to uppercase or lowercase to prevent case-sensitive lookup bugs.
- Schema validation: raise an error if a value does not conform to an expected type or range.
- Read-only dictionaries: raise an error in `__setitem__` to prevent any modifications after construction.
- Audit dictionaries: log every write operation for debugging or compliance purposes.

`__setitem__` vs. `update()`

When you override `__setitem__` in a UserDict subclass, calling `d.update({'key': 'value'})` or `d['key'] = 'value'` will both invoke your override. This is the main advantage of UserDict over a raw dict subclass — with a raw dict, `update()` may bypass `__setitem__` in some Python implementations.

1.11 UserString — Custom String Classes

UserString wraps a regular string stored in `self.data` and allows you to extend or override any string operation. Unlike lists and dictionaries, strings are immutable — you cannot change their content in place. UserString subclasses are therefore used to add new methods or modify the return type of existing operations, rather than to add write-time validation.

Common use cases for UserString:

- Domain-specific string types: a `SQLQuery` class that validates syntax before execution, or a `FilePath` class that normalizes separators.
- Augmented formatting: add a `shout()` method that returns the string in uppercase, or a `slugify()` method that converts text to URL-safe slugs.
- Wrapped metadata: attach additional attributes (like a language code or encoding type) to string instances.

1.12 Benefits of Custom Container Classes

Creating custom containers by subclassing `UserList`, `UserDict`, or `UserString` is a form of domain modelling — you are encoding the rules and invariants of your specific problem domain into the data structure itself, rather than scattering validation checks throughout your codebase.

The key benefits are:

- Encapsulation of validation: The rule 'only positive numbers are allowed' lives in the container class, not duplicated in every place that adds to the list.

- Single responsibility: Business logic that governs data validity belongs with the data structure.
- Code reuse: Once written and tested, the custom container can be used anywhere in the codebase without repeating guards.
- Readability: `PositiveNumberList.append(x)` is far more expressive than `[list.append(x) for x in values if x > 0]`.
- Maintainability: Changing a validation rule requires a change in one place — the class definition.
- Safety: Invalid data is rejected at the point of entry, not discovered later as a confusing downstream bug.

SECTION 2 — PRACTICAL & CODING (IPYTHON NOTEBOOK WALKTHROUGH)

Each cell below is presented exactly as it appears in a Jupyter notebook, followed by a comprehensive explanation covering what the code does, the key concepts it demonstrates, what output to expect, and how the pattern applies in real data science and engineering work.

 How to Follow Along

Create a new Jupyter notebook, name it `python_collections.ipynb`, and run each cell in sequence. After running each cell, read the explanation below it before proceeding to the next. Actively modify values and re-run — that hands-on loop is the fastest path to mastery.

Cell 1 — The Four Container Types at a Glance

In [1]:

```
numbers = [1, 2, 3]
person   = ("John", 25)
colors   = {"red", "blue"}
student  = {"name": "Alice"}

print(type(numbers))
print(type(person))
print(type(colors))
print(type(student))
```

Explanation:

This cell creates one instance of each of Python's four built-in container types and prints the type of each. It is the canonical starting point for understanding what makes each container distinct.

`numbers = [1, 2, 3]`: A list literal. Square brackets define a list. The three integers are stored in order at indices 0, 1, and 2.

`person = ("John", 25)`: A tuple literal. Parentheses define a tuple. Notice that the tuple holds values of different types — a string and an integer — illustrating that containers are heterogeneous.

`colors = {"red", "blue"}`: A set literal. Curly braces with no colons define a set. If you added "red" a second time, the set would still contain only one "red" — duplicates are silently ignored.

`student = {"name": "Alice"}`: A dictionary literal. Curly braces with key: value pairs define a dictionary. The string "name" is the key, and "Alice" is its associated value.

Expected output: `<class 'list'>`, `<class 'tuple'>`, `<class 'set'>`, `<class 'dict'>`. The `type()` built-in function confirms that Python treats these as four distinct types, each with its own class and set of methods.

Cell 2 — List Mutation: Adding and Sorting

In [2]:

```
fruits = ["apple", "banana", "orange"]
fruits.append("grape")
print(fruits)
```

Explanation:

This cell demonstrates the two most fundamental aspects of lists: that they store items in order and that they can be changed after creation (mutability).

`fruits = ["apple", "banana", "orange"]`: Creates a list of three string elements. The order is preserved exactly as written — apple is at index 0, banana at index 1, orange at index 2.

`fruits.append("grape")`: The `append()` method adds "grape" to the end of the list. This is an in-place operation — it modifies `fruits` directly and returns `None`. After this call, the list has four elements.

`print(fruits)`: Prints the full list, including the newly appended element.

Expected output: `['apple', 'banana', 'orange', 'grape']`

Notice that the original three elements retain their positions. `append()` never reorders or shifts existing elements — it always places the new item at `index len(fruits) - 1` after the operation.

Real-world pattern: In a data ingestion pipeline, you might start with an empty list `results = []` and call `results.append(process(record))` inside a loop to accumulate processed records before converting the entire list to a pandas DataFrame.

Cell 3 — Sorting a List In Place

In [3]:

```
numbers = [3, 1, 4, 1, 5, 9, 2, 6]
numbers.sort()
print(numbers)
```

Explanation:

This cell demonstrates the `sort()` method, which rearranges all elements of a list in ascending order in place using Python's TimSort algorithm (a hybrid of merge sort and insertion sort that is highly efficient on real-world data).

`numbers = [3, 1, 4, 1, 5, 9, 2, 6]`: Creates an unsorted list with a duplicate (1 appears twice). Notice that duplicates are perfectly valid in lists.

`numbers.sort()`: Sorts the list in place in ascending order. Like `append()`, this modifies the original list and returns `None` — it does NOT create a new list. If you need the original order preserved, use the built-in `sorted(numbers)` function instead, which returns a new sorted list without modifying the original.

Expected output: `[1, 1, 2, 3, 4, 5, 6, 9]`. Duplicates are preserved in the sorted output.

Key distinction to remember: `list.sort()` modifies in place and returns `None`. `sorted(iterable)` returns a new sorted list and leaves the original unchanged. Use `sort()` when you do not need the original order; use `sorted()` when you need both.

For descending order, use `numbers.sort(reverse=True)`. For custom sorting logic, use the `key` parameter: `words.sort(key=len)` sorts strings by length.

Cell 4 — List Comprehensions for Transformation

In [4]:

```
squares = [x * x for x in range(5)]  
print(squares)
```

Explanation:

This cell introduces list comprehensions — one of Python's most elegant and widely used features. This single line replaces what would otherwise be a four-line for-loop with `append()`.

`range(5)`: Generates the sequence of integers 0, 1, 2, 3, 4. `range()` returns an iterator, not a list, which is memory-efficient — it produces one value at a time rather than allocating all five integers simultaneously.

`x * x for x in range(5)`: For each value `x` produced by `range(5)`, this expression computes `x` squared (`x * x`). The results are: `0*0=0`, `1*1=1`, `2*2=4`, `3*3=9`, `4*4=16`.

The outer square brackets collect all the computed values into a new list. The original `range` is not modified.

Expected output: `[0, 1, 4, 9, 16]`

Equivalent for-loop for comparison:

Equivalent for-loop (less Pythonic)

```
squares = []  
for x in range(5):  
    squares.append(x * x)  
print(squares)  # [0, 1, 4, 9, 16]
```

List comprehensions with a filter condition are equally powerful:

Comprehension with filter — only even squares

```
even_squares = [x * x for x in range(10) if x % 2 == 0]  
print(even_squares)  # [0, 4, 16, 36, 64]
```

Cell 5 — Tuple Creation and Index Access

In [5]:

```
person = ("John", 25, "Developer")  
print(person[0])
```

Explanation:

This cell creates a tuple and accesses one of its elements by index. The syntax for index access on a tuple is identical to that of a list — square brackets with the integer index.

`person = ("John", 25, "Developer")`: Creates a three-element tuple containing a string, an integer, and another string. The parentheses define the tuple literal.

`person[0]`: Retrieves the element at index 0, which is "John". Tuples are zero-indexed, exactly like lists.

Expected output: John

What happens if you try to modify a tuple?

Attempting to modify a tuple raises `TypeError`

```
person[0] = "Jane"
# TypeError: 'tuple' object does not support item assignment
```

This immutability is intentional and beneficial. In a data pipeline, if a function returns (`processed_data`, `record_count`, `elapsed_seconds`) as a tuple, downstream code can trust that no one has accidentally modified those values while passing them around.

Cell 6 — Tuple Unpacking

In [6]:

```
person = ("John", 25)
name, age = person
print(name)
print(age)
```

Explanation:

This cell demonstrates tuple unpacking — one of Python's most expressive and frequently used features. Unpacking assigns the elements of a tuple (or any iterable) to multiple variables in a single statement.

`name, age = person`: The left side has two variable names separated by a comma. Python unpacks the two-element tuple `person` and assigns "John" to `name` and 25 to `age` simultaneously.

This works because Python counts the elements: the tuple has 2 elements, and there are 2 variables on the left. If the counts do not match, a `ValueError` is raised.

Expected output: John (first print), then 25 (second print).

Extended unpacking with the star operator is also supported:

Extended unpacking with `*`

```
first, *middle, last = (1, 2, 3, 4, 5)
print(first)      # 1
print(middle)     # [2, 3, 4]
print(last)       # 5
```

Tuple unpacking is especially useful when iterating over a dictionary's items():

Unpacking in for loops

```
student = {"name": "Alice", "grade": "A", "score": 95}
for key, value in student.items():
    print(f"{key}: {value}")
```

Cell 7 — Sets and Automatic Deduplication

In [7]:

```
numbers = {1, 2, 3, 3, 2}
print(numbers)
```

Explanation:

This cell demonstrates the defining property of sets: they automatically eliminate duplicate values. No matter how many times you include a value in the set literal, it will appear only once.

`numbers = {1, 2, 3, 3, 2}`: The set literal contains five values, but 2 and 3 each appear twice. Python silently ignores the duplicates at construction time.

Expected output: `{1, 2, 3}` — though the exact order of elements printed may vary between runs, because sets are unordered. Do not write code that depends on a particular iteration order of a set.

A highly practical pattern — converting a list to a set to deduplicate it:

Deduplication pattern — list to set and back

```
raw_tags = ["python", "ml", "python", "ai", "ml", "python"]
unique_tags = list(set(raw_tags))
print(unique_tags) # ['ai', 'ml', 'python'] (order may vary)
```

Important: converting to a set loses the original order. If you need to deduplicate while preserving order, use a different pattern:

Order-preserving deduplication (Python 3.7+)

```
unique_ordered = list(dict.fromkeys(raw_tags))
print(unique_ordered) # ['python', 'ml', 'ai']
```

Cell 8 — Set Operations: Intersection, Union, Difference

In [8]:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
print(set1.intersection(set2))
```

Explanation:

This cell demonstrates Python's built-in set operations, which mirror the mathematical concepts of set theory. These operations are extremely efficient — they run in $O(\min(\text{len}(s1), \text{len}(s2)))$ time for intersection.

`set1.intersection(set2)`: Returns a new set containing only the elements that appear in both `set1` and `set2`. Here, 3 is the only value present in both.

Expected output: {3}

All four fundamental set operations are available:

All four set operations demonstrated		
<code>set1 = {1, 2, 3, 4}</code>		
<code>set2 = {3, 4, 5, 6}</code>		
<code>print(set1.union(set2))</code> from both	# {1, 2, 3, 4, 5, 6}	— all elements
<code>print(set1.intersection(set2))</code> in BOTH	# {3, 4}	— only elements
<code>print(set1.difference(set2))</code> NOT in set2	# {1, 2}	— in set1 but
<code>print(set1.symmetric_difference(set2))</code> NOT in both	# {1, 2, 5, 6}	— in either but

Real-world use case: You have two lists of user IDs — users who opened an email (`set_opened`) and users who clicked a link (`set_clicked`). The users who opened but did not click are: `set_opened.difference(set_clicked)`. Those who did both are: `set_opened.intersection(set_clicked)`.

Cell 9 — Dictionary Creation and Value Access

In [9]:
<code>student = {"name": "Alice", "age": 20}</code>
<code>print(student["name"])</code>

Explanation:

This cell creates a dictionary and retrieves a value by its key. Dictionary lookup by key is one of the most common operations in Python programming — it underlies JSON parsing, configuration management, feature extraction, and much more.

`student = {"name": "Alice", "age": 20}`: Creates a dictionary with two key-value pairs. The keys are strings ("name" and "age"), and the values are a string ("Alice") and an integer (20).

`student["name"]`: Retrieves the value associated with the key "name". Because dictionaries use hash tables internally, this lookup is $O(1)$ — it takes the same time whether the dictionary has 2 entries or 2 million.

Expected output: Alice

What happens with a missing key?

Safe key access with .get()

```
# This raises KeyError:
# print(student["grade"])

# Safe alternative — returns None if key not found:
grade = student.get("grade")
print(grade)  # None

# With a default value:
grade = student.get("grade", "N/A")
print(grade)  # N/A
```

Always prefer `.get()` over direct bracket access when a key may or may not be present, especially when processing API responses or user-provided data.

Cell 10 — Adding and Updating Dictionary Entries**In [10]:**

```
student = {"name": "Alice"}
student["age"] = 20
print(student)
```

Explanation:

This cell demonstrates dynamic mutation of a dictionary. Unlike tuples (immutable) and unlike sets (which have a specific interface for adding elements), dictionaries are modified using the same square-bracket syntax used for access.

`student["age"] = 20`: If the key "age" does not exist in the dictionary, this statement adds a new key-value pair. If "age" already existed, this statement would overwrite its value. Both cases use identical syntax — Python handles them the same way.

Expected output: `{'name': 'Alice', 'age': 20}`. Notice that the output preserves insertion order: 'name' was inserted first (in the literal), 'age' second.

Updating multiple keys at once using `update()`:

Bulk update with .update()

```
student.update({"grade": "A", "score": 95, "name": "Alice M."})
print(student)
# {'name': 'Alice M.', 'age': 20, 'grade': 'A', 'score': 95}
# Note: 'name' was overwritten from 'Alice' to 'Alice M.'
```

Cell 11 — UserList: Enforcing Constraints on List Content**In [11]:**

```
from collections import UserList

class PositiveNumbers(UserList):
    def append(self, item):
        if item > 0:
            super().append(item)

nums = PositiveNumbers()
nums.append(5)
nums.append(-3)    # silently ignored
nums.append(10)
print(nums)
```

Explanation:

This is the first custom container cell and one of the most important in this chapter. It shows how to create a list that enforces a business rule at the data-structure level, rather than scattering guard clauses throughout application code.

`from collections import UserList`: Imports the `UserList` wrapper from Python's standard library.

`class PositiveNumbers(UserList)`: Defines a new class that inherits from `UserList`. `PositiveNumbers` IS a list in every way — it supports all list methods and operations — except that its `append()` method has been overridden.

`def append(self, item)`: Overrides the `append()` method. This is called every time code calls `nums.append(something)`.

`if item > 0: super().append(item)`: Only if the item is strictly positive does it call the parent class's `append()` (i.e., the real list's `append`). Negative numbers and zero are silently rejected.

`nums.append(-3)`: This does nothing — the condition `item > 0` is `False` for `-3`, so `super().append()` is never called.

Expected output: `[5, 10]` — only the two positive values that passed the validation.

This pattern — validation at the container level — is valuable in data pipelines where you want to guarantee that a collection only ever holds valid data, making downstream processing simpler and safer.

Extended example — type-checked list

```
from collections import UserList

class StringOnlyList(UserList):
    def append(self, item):
        if not isinstance(item, str):
            raise TypeError(f"Expected str, got {type(item).__name__}")
        super().append(item)

tags = StringOnlyList()
```

```
tags.append("machine-learning")
tags.append("python")
# tags.append(42) # would raise: TypeError: Expected str, got int
print(tags) # ['machine-learning', 'python']
```

Cell 12 — UserDict: Normalizing Dictionary Keys

In [12]:

```
from collections import UserDict

class UpperDict(UserDict):
    def __setitem__(self, key, value):
        super().__setitem__(key.upper(), value)

data = UpperDict()
data["name"] = "John"
data["City"] = "Delhi"
print(data)
```

Explanation:

This cell creates a custom dictionary that automatically normalizes all keys to uppercase at write time. This is a real-world pattern used to prevent case-sensitivity bugs in configuration parsers, HTTP header handling (HTTP headers are case-insensitive), and data normalization pipelines.

`class UpperDict(UserDict)`: Inherits from `UserDict`. All standard dictionary behaviour is inherited.

`def __setitem__(self, key, value)`: This special method (a dunder method) is called whenever a key-value pair is assigned — both for new entries (`data["name"] = "John"`) and for updates to existing keys.

`super().__setitem__(key.upper(), value)`: Calls the parent class's `__setitem__` with the uppercased key. The value is stored unchanged.

`data["City"] = "Delhi"`: Even though the key is written as "City" (mixed case), it is stored as "CITY" due to the override.

Expected output: `{'NAME': 'John', 'CITY': 'Delhi'}`

This means you can now look up values using any case: `data["name"]`, `data["NAME"]`, `data["Name"]` will all work, because all keys are stored in uppercase. However, you must also override `__getitem__` if you want lookups to be case-insensitive as well:

Fully case-insensitive dictionary

```
from collections import UserDict

class CaseInsensitiveDict(UserDict):
    def __setitem__(self, key, value):
        super().__setitem__(key.lower(), value)
```

```
def __getitem__(self, key):
    return super().__getitem__(key.lower())

def get(self, key, default=None):
    return super().get(key.lower(), default)

config = CaseInsensitiveDict()
config["Host"] = "localhost"
print(config["HOST"])    # localhost
print(config["host"])    # localhost
```

Cell 13 — UserString: Adding Custom Methods to Strings

In [13]:

```
from collections import UserString

class ShoutString(UserString):
    def shout(self):
        return self.data.upper()

text = ShoutString("hello world")
print(text.shout())
print(text.upper())    # inherited from regular string
print(len(text))       # inherited — works as expected
```

Explanation:

This cell creates a custom string class that adds a new method (`shout()`) while retaining all the normal string methods through inheritance from `UserString`.

`class ShoutString(UserString)`: Inherits from `UserString`. The wrapped string is stored in `self.data`.

`def shout(self)`: A completely new method that does not exist on regular Python strings. It returns the string converted to uppercase using `self.data.upper()`.

`text = ShoutString("hello world")`: Creates an instance. `text` behaves like a regular string in every way — it supports `upper()`, `lower()`, `split()`, `strip()`, `len()`, slicing, concatenation, and so on.

`text.shout()`: Calls the custom method. Expected output: `HELLO WORLD`

`text.upper()`: Calls the inherited `upper()` method from the regular string interface. Expected output: `HELLO WORLD` (same result here, but `shout()` could do something more complex).

A more practical example — a URL-safe slug generator:

Practical UserString example — Slug generator

```
from collections import UserString
import re
```

```
class SlugString(UserString):
    def slugify(self):
        # Convert to lowercase, replace spaces and special chars with
        hyphens
        slug = self.data.lower()
        slug = re.sub(r"[^a-z0-9]+", "-", slug)
        return slug.strip("-")

title = SlugString("Python Collections: A Complete Guide!")
print(title.slugify())    # python-collections-a-complete-guide
```

Cell 14 — Composing Validation Logic in Custom Containers

In [14]:

```
from collections import UserList

class EvenList(UserList):
    def append(self, item):
        if item % 2 == 0:
            super().append(item)

evens = EvenList()
for n in range(10):
    evens.append(n)

print(evens)
```

Explanation:

This final cell reinforces the `UserList` pattern with a slightly more complex example and combines it with a for-loop to show how custom containers work naturally in typical Python iteration patterns.

`class EvenList(UserList)`: A list that accepts only even numbers. The validation rule is encapsulated in the `append()` override: if `item % 2 == 0` (i.e., item is divisible by 2), it is accepted; otherwise it is silently discarded.

`for n in range(10)`: Iterates over 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 and calls `evens.append(n)` for each. The `EvenList` filter runs on every call — odd numbers (1, 3, 5, 7, 9) are rejected by the if condition.

Expected output: [0, 2, 4, 6, 8]

This demonstrates a key architectural insight: by encoding the business rule inside the container, the for-loop itself remains completely clean and simple — it just appends every number without needing to know the filtering logic. This separation of concerns is the hallmark of good software design.

You could make the filter configurable at construction time for maximum reusability:

Configurable predicate list — reusable pattern

```

from collections import UserList

class FilteredList(UserList):
    def __init__(self, predicate, initial_data=None):
        super().__init__(initial_data or [])
        self.predicate = predicate

    def append(self, item):
        if self.predicate(item):
            super().append(item)

# Usage examples
positives = FilteredList(lambda x: x > 0)
evens      = FilteredList(lambda x: x % 2 == 0)
long_words = FilteredList(lambda s: len(s) > 5)

for n in range(-3, 6):
    positives.append(n)
    evens.append(n)

print(positives)  # [1, 2, 3, 4, 5]
print(evens)     # [0, 2, 4]

```

Chapter Summary — Quick Reference

The following table consolidates the most important patterns and method signatures from this chapter:

Pattern	Code	Notes
Create list	<code>lst = [1, 2, 3]</code>	Ordered, mutable
List comprehension	<code>[x*2 for x in lst if x > 0]</code>	Fast, Pythonic
Create tuple	<code>t = (1, 2, 3)</code>	Ordered, immutable
Tuple unpacking	<code>a, b, c = t</code>	Assigns each element
Create set	<code>s = {1, 2, 3}</code>	Unique, unordered
Deduplicate list	<code>list(set(lst))</code>	Loses original order
Set intersection	<code>s1 & s2</code> or <code>s1.intersection(s2)</code>	Elements in both
Create dict	<code>d = {'k': 'v'}</code>	Key-value mapping
Safe dict access	<code>d.get('k', default)</code>	No <code>KeyError</code>
Iterate dict	<code>for k, v in d.items():</code>	Key and value

Custom list	<code>class C(UserList): ...</code>	Override <code>append()</code>
Custom dict	<code>class C(UserDict): ...</code>	Override <code>__setitem__</code>
Custom string	<code>class C(UserString): ...</code>	Add new methods

What's Next

Chapter 3 covers advanced Python data structures and the full `collections` module in depth: `namedtuple` for readable records, `defaultdict` for auto-initializing keys, `Counter` for frequency analysis, `deque` for efficient queues, and `ChainMap` for layered configuration — along with practical applications in data processing pipelines.